ACTIVITY LAB 3: SHELL PROGRAMMING

UNIX System V Bourne Shell (/bin/sh)

Student Name	Martin, Rodnee Glen
Year & Section	3 BSCS A
Operating System	AT&T UNIX System V Release 4 Version 2.1 (3 1/2)
System Nodename	MartinRodneeGlen
Shell Used	/bin/sh (System V Bourne shell)
Hypervisor	Oracle VM VirtualBox 7.2
Date Performed	Thursday, September 3, 2026 (22:32 - 22:47)
Working Directory	/lab3

Answer Summary

#	Required Activity	Shell Command
1	Print “Hello Welcome to shell Programming”	**echo**
2	Get the value from the user	**read**
3	Make a variable global	**export**
4	Perform a numeric operation	**expr**
5	Make use of command substitution	**`command` (back quotes)**

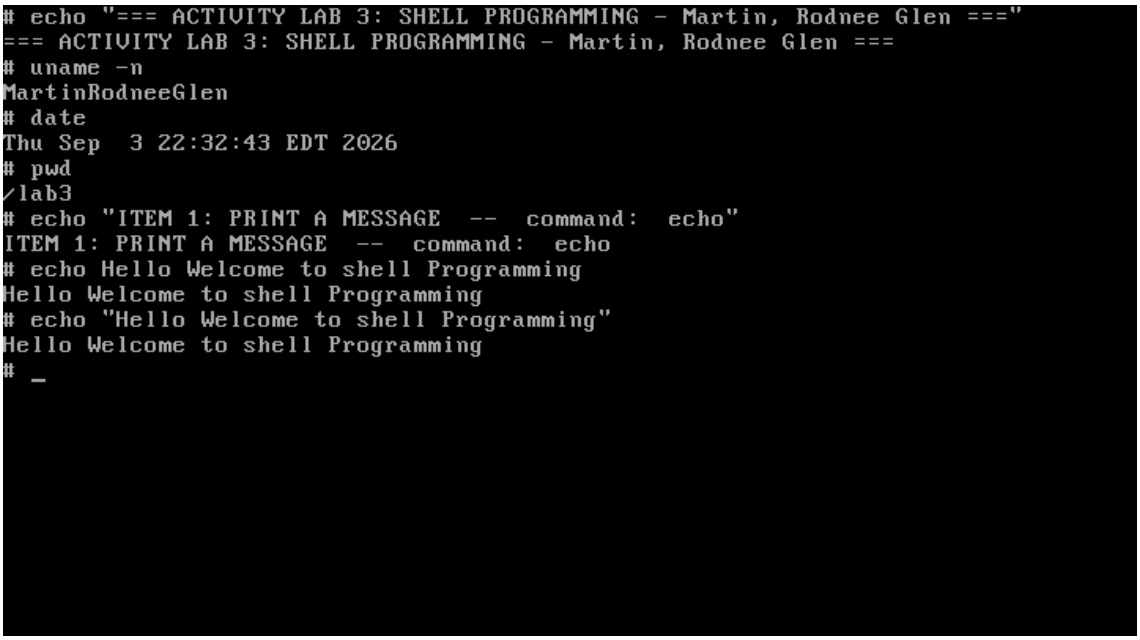
How the activity was performed

- Every command was executed live in the AT&T UNIX System V Release 4 virtual machine, logged in as root, inside the working directory /lab3.
- Items 1 to 5 were first demonstrated one by one at the shell prompt (screenshots 1-5).
- The five items were then combined into one executable shell script, lab3.sh, which was created with cat > lab3.sh, made executable with chmod +x and executed (screenshots 6-7).
- Each screenshot below shows both the command/script and its resulting output.

ITEM 1 | Print a Message - command: echo

Commands executed

```
# uname -n
# date
# pwd
# echo Hello Welcome to shell Programming
# echo "Hello Welcome to shell Programming"
```



Screenshot 1 - the required message printed with echo, together with the system name, date and working directory.

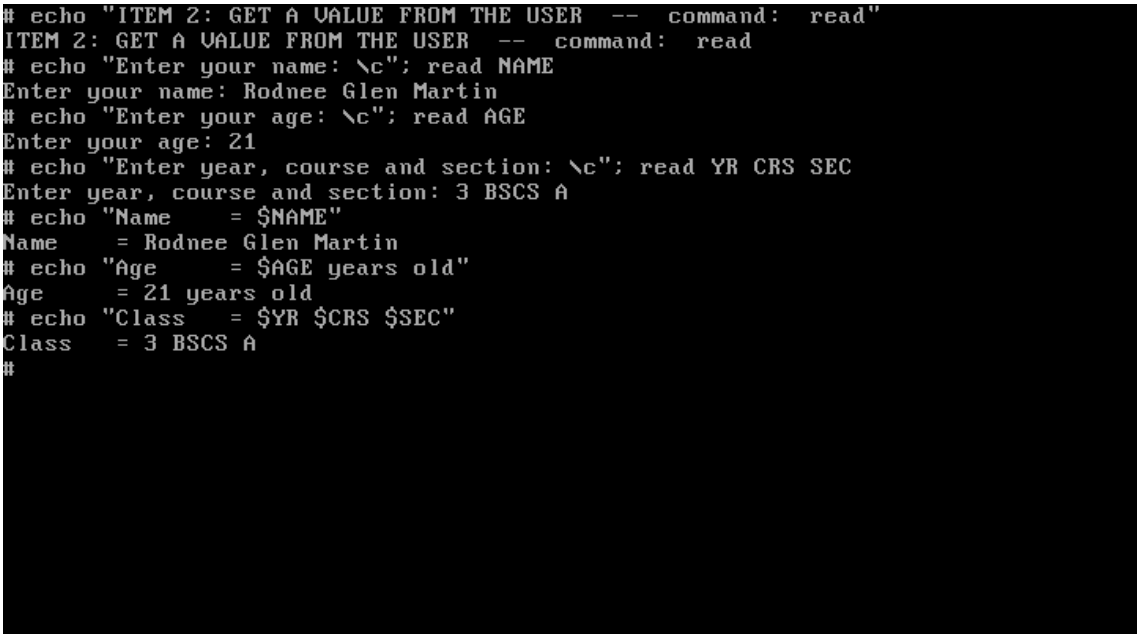
Explanation

- echo writes its arguments to standard output followed by a newline.
- Both the unquoted and the quoted form print the same message.
- Quoting is the safer habit: it keeps the message as ONE argument and protects characters such as *, ?, > and repeated spaces from being interpreted by the shell.

ITEM 2 | Get a Value From the User - command: `read`

Commands executed

```
# echo "Enter your name: \c"; read NAME
Enter your name: Rodnee Glen Martin
# echo "Enter your age: \c"; read AGE
Enter your age: 21
# echo "Enter year, course and section: \c"; read YR CRS SEC
Enter year, course and section: 3 BSCS A
# echo "Name    = $NAME"
# echo "Age     = $AGE years old"
# echo "Class   = $YR $CRS $SEC"
```



Screenshot 2 - read stores keyboard input in a variable, which is then displayed with echo.

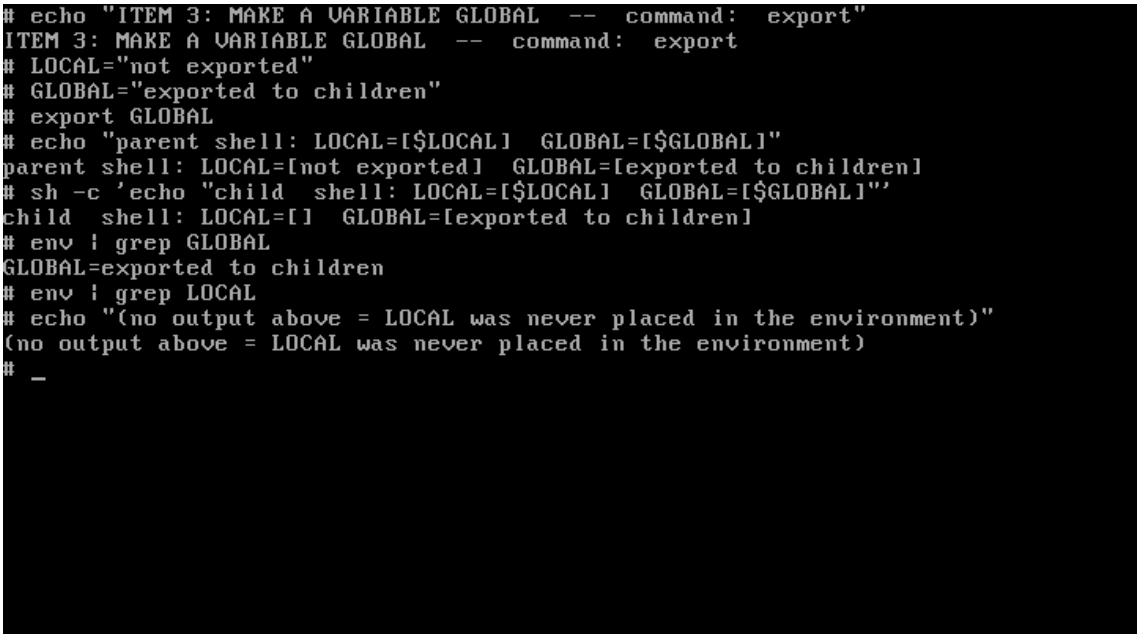
Explanation

- read VAR pauses the shell, reads one line from the keyboard and stores it in VAR; the value is used afterwards as $VAR.
- read accepts several variable names at once, so the words of one input line are assigned in order (3 to YR, BSCS to CRS, A to SEC). If there are more words than variables, the last variable receives all the remaining words.
- In System V, \c inside echo suppresses the newline so the answer is typed on the same line as the prompt (the System V equivalent of echo -n).

ITEM 3 | Make a Variable Global - command: `export`

Commands executed

```
# LOCAL="not exported"
# GLOBAL="exported to children"
# export GLOBAL
# echo "parent shell: LOCAL=[$LOCAL]  GLOBAL=[$GLOBAL]"
# sh -c 'echo "child  shell: LOCAL=[$LOCAL]  GLOBAL=[$GLOBAL]"'
# env | grep GLOBAL
# env | grep LOCAL
```



Screenshot 3 - the child shell inherits GLOBAL because it was exported, while LOCAL is empty there and never appears in env.

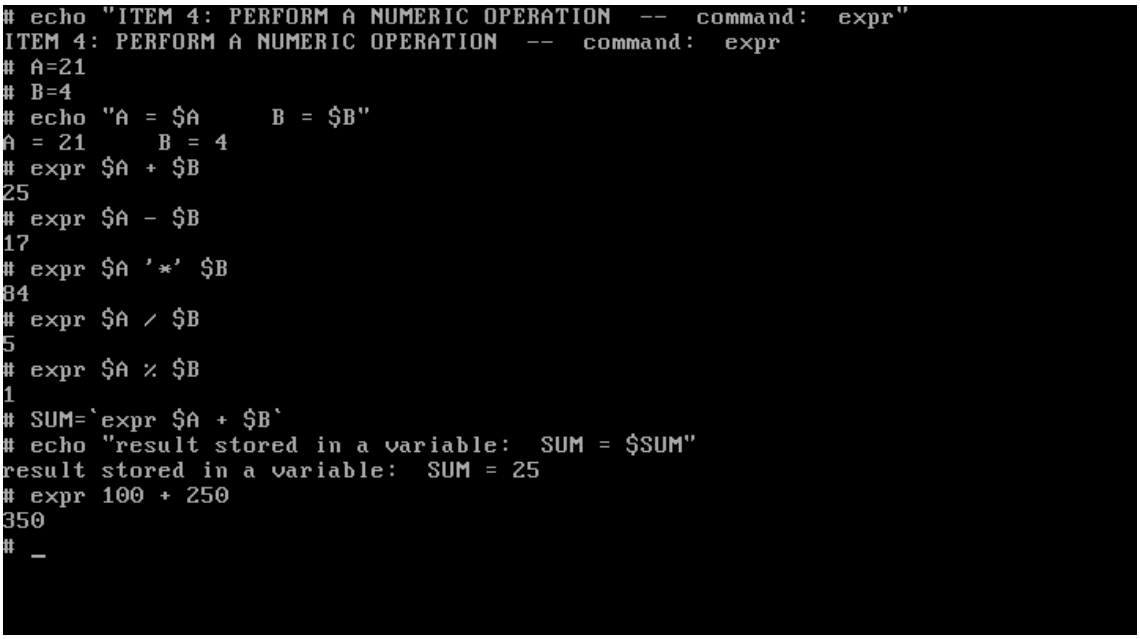
Explanation

- A plain assignment creates a variable that lives only in the current shell.
- export NAME places the variable in the environment, so every child process - here the child shell started by sh -c - inherits a copy of it.
- Proof: inside the child shell $GLOBAL still has its value while $LOCAL is empty, and env lists GLOBAL but never LOCAL.
- Bourne shell syntax is two steps: NAME=value first, then export NAME.
- Export is one-way: a child may read the value, but changes made inside the child never travel back to the parent shell.

ITEM 4 | Perform a Numeric Operation - command: expr

Commands executed

```
# A=21
# B=4
# expr $A + $B          ->  25    addition
# expr $A - $B          ->  17    subtraction
# expr $A '*' $B        ->  84    multiplication
# expr $A / $B          ->   5    integer division
# expr $A % $B          ->   1    remainder (modulo)
# SUM=`expr $A + $B`
# echo "result stored in a variable:  SUM = $SUM"
# expr 100 + 250        -> 350
```



Screenshot 4 - the five arithmetic operations evaluated by expr, including a result stored in the variable SUM.

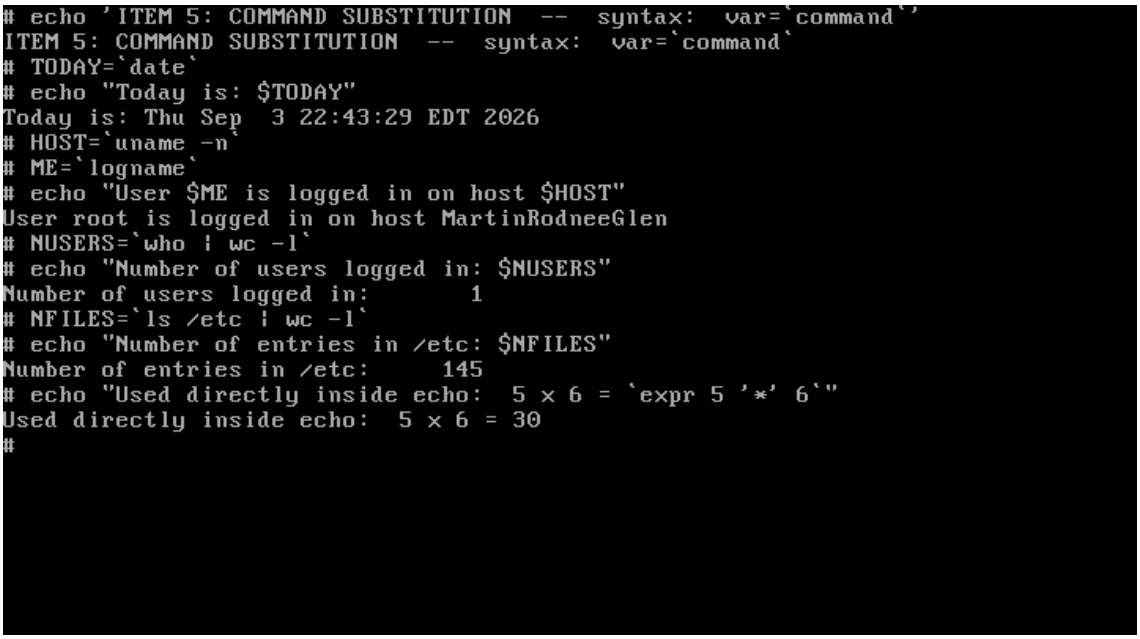
Explanation

- The shell keeps every value as text, so arithmetic is performed by the external command expr, which evaluates the expression and prints the result.
- Every operand and every operator must be a separate argument: expr 21 + 4 works, while expr 21+4 only prints the string 21+4.
- The * must be quoted ('*' or *), otherwise the shell would first expand it into the list of file names in the current directory before expr ever sees it.
- expr is integer only, so 21 / 4 gives 5 and 21 % 4 gives the remainder 1.
- Wrapping expr in back quotes stores the answer in a variable for later use.

ITEM 5 | Command Substitution - syntax: `var=`command``

Commands executed

```
# TODAY=`date`
# echo "Today is: $TODAY"
# HOST=`uname -n`
# ME=`logname`
# echo "User $ME is logged in on host $HOST"
# NUSERS=`who | wc -l`
# echo "Number of users logged in: $NUSERS"
# NFILES=`ls /etc | wc -l`
# echo "Number of entries in /etc: $NFILES"
# echo "Used directly inside echo:  5 x 6 = `expr 5 '*' 6`"
```



Screenshot 5 - the output of date, uname, logname and of a whole pipeline substituted into variables and into an echo string.

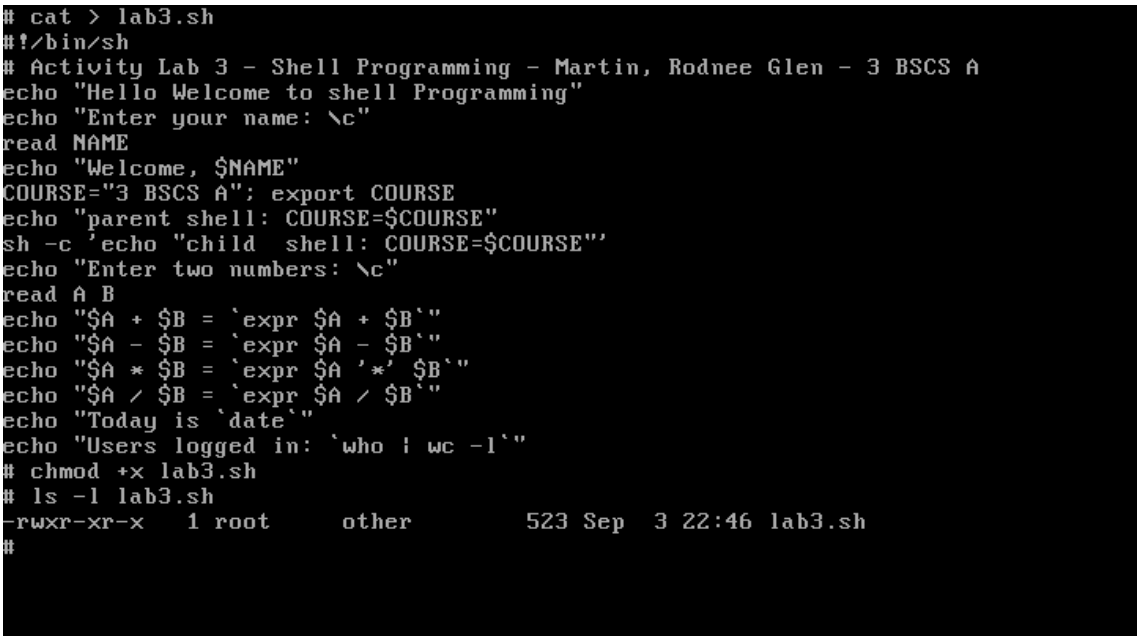
Explanation

- Text placed between back quotes is executed first, and its standard output replaces the back-quoted text on the command line.
- The result can be assigned to a variable (TODAY=`date`) or used straight inside a double-quoted string, as in the last echo.
- A whole pipeline may be substituted, for example `who | wc -l` returns the number of users currently logged in.
- Note: the modern $(command) form does not exist in this shell. On this System V Bourne shell, echo $(date) returns syntax error: `(' unexpected , so back quotes are the correct syntax for this environment.

SHELL SCRIPT | Creating `lab3.sh`

Commands executed

```
# cat > lab3.sh          (script typed in, ended with Ctrl+D)
# chmod +x lab3.sh
# ls -l lab3.sh
-rwxr-xr-x   1 root     other        523 Sep  3 22:46 lab3.sh
```



Screenshot 6 - the script is created with cat > lab3.sh, then made executable with chmod +x (see the x bits in -rwxr-xr-x).

Script source (lab3.sh)

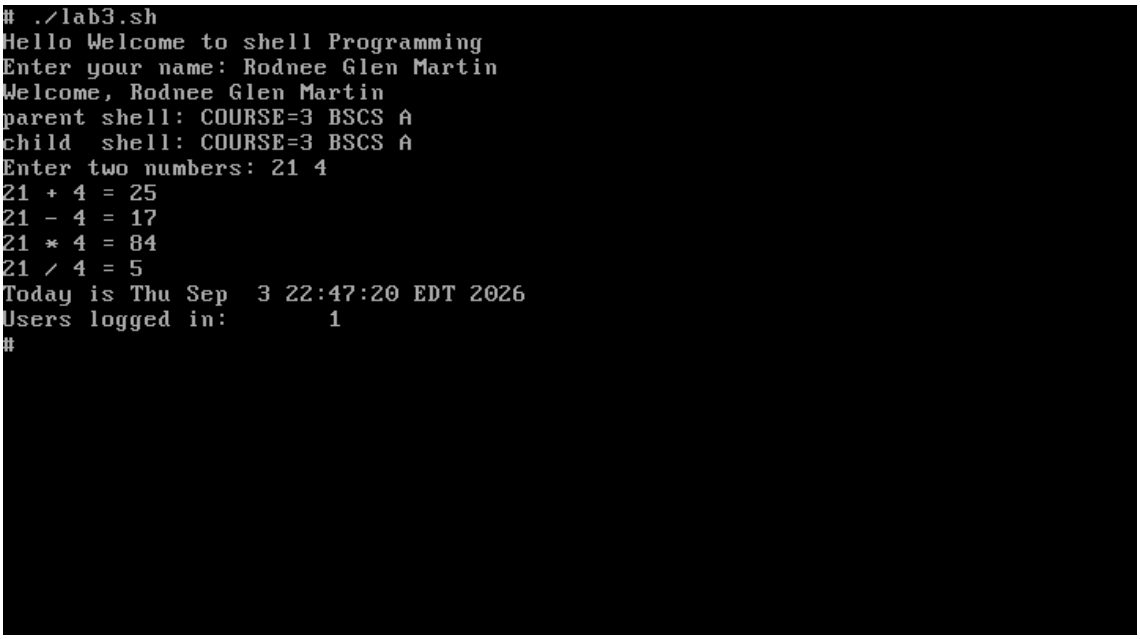
```
#!/bin/sh
# Activity Lab 3 - Shell Programming - Martin, Rodnee Glen - 3 BSCS A
echo "Hello Welcome to shell Programming"          <-- item 1  echo
echo "Enter your name: \c"                         <-- item 2  read
read NAME
echo "Welcome, $NAME"
COURSE="3 BSCS A"; export COURSE                   <-- item 3  export
echo "parent shell: COURSE=$COURSE"
sh -c 'echo "child  shell: COURSE=$COURSE"'
echo "Enter two numbers: \c"                       <-- item 4  expr
read A B
echo "$A + $B = `expr $A + $B`"
echo "$A - $B = `expr $A - $B`"
echo "$A * $B = `expr $A '*' $B`"
echo "$A / $B = `expr $A / $B`"
echo "Today is `date`"                            <-- item 5  `cmd`
echo "Users logged in: `who | wc -l`"
```

#!/bin/sh on the first line tells the kernel which shell must interpret the file; the lines beginning with # are comments. The arrows on the right are added here only to map each line to the required item.

SHELL SCRIPT | Executing `./lab3.sh`

Command executed and full output

```
# ./lab3.sh
Hello Welcome to shell Programming
Enter your name: Rodnee Glen Martin
Welcome, Rodnee Glen Martin
parent shell: COURSE=3 BSCS A
child  shell: COURSE=3 BSCS A
Enter two numbers: 21 4
21 + 4 = 25
21 - 4 = 17
21 * 4 = 84
21 / 4 = 5
Today is Thu Sep  3 22:47:20 EDT 2026
Users logged in:        1
```



Screenshot 7 - one run of lab3.sh demonstrating all five required items.

Explanation

- A single run of the script covers every requirement: echo prints the required message, read collects the name and the two numbers, export passes COURSE to the child shell, expr computes the four results, and the back quotes substitute the output of date and of the who | wc -l pipeline.
- ./lab3.sh runs the file because of the execute bit added by chmod; sh lab3.sh would also run it even without that permission.